ДОМАШНЕЕ ЗАДАНИЕ №2

ZerO: нужен ли нейросети рандом на старте?

Разбор статьи, воспроизведение и дополнительный опыт

Веса нейросети перед обучением обычно заполняют случайными числами. Авторы ZerO решили проверить: а рандом здесь вообще обязателен? В их опытах удалось обучить сети с заранее заданными весами [1].

Веса - коэффициенты, с которыми слой складывает входные числа. Инициализация задает их до обучения, а матрица весов - просто таблица этих коэффициентов. Давайте попробуем заполнить ее по заранее выбранному правилу.

Почему не поставить везде нули? В этой многослойной сети без добавочных смещений, bias, тогда скрытые признаки и градиенты весов тоже нулевые. Обучение застревает. Это подтверждается контрольным запуском.

Другой вариант - **identity**, единичная матрица: каждый выход копирует свою входную координату. Но при расширении слоя копировать на новые места нечего:

`(2, 5) → (2, 5, 0, 0)`

В моем опыте с ReLU, без bias и дополнительного уменьшения весов (weight decay), новые координаты так и остаются нулевыми. Я добавил нейроны, но эта часть сети не работает. Идея ZerO - заранее смешать вход, чтобы значения попали и в новые координаты.

Смешиваем входы без рандома

Давайте возьмем два числа и посчитаем сумму и разность. Из (2,1) получим (3,1). Вот простейшая **матрица Хадамарда**: таблица коэффициентов +1 и −1, которая задает такие комбинации.

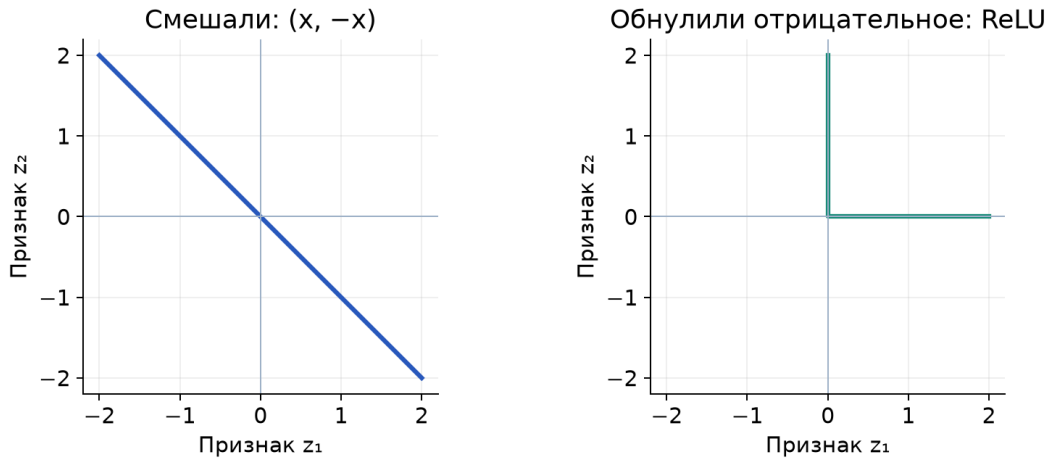
Коэффициенты	Что считаем
(+1, +1)	a+b
(+1, −1)	a−b

Большие таблицы строятся по тому же повторяющемуся рисунку знаков. Получаются суммы и разности входов, для которых не понадобилось ни одного случайного числа. В ZerO такой прием используется, когда слой расширяется.

Источник: Zhao, Schäfer, Anandkumar. ZerO Initialization, TMLR 2022, arXiv v3 [1].

1. Как это помогает сети

Одного смешивания мало: линейные операции подряд остаются линейными. Между слоями стоит **ReLU** - отрицательные значения обнуляются. На разных входах она оставляет разные комбинации.



Учебный пример x→(x,−x): после ReLU одна прямая превращается в два луча. Это иллюстрация механизма, а не буквальный слой ZerO 1→2.

Входное число по-прежнему одно. Но признаки для разных входов теперь лежат на двух осях, а не на одной прямой. Количество независимых направлений описывает **ранг**. Так ReLU может расширить набор направлений в признаках, хотя новой информации во входе не появилось.

Размер слоя	Рецепт Algorithm 1
Не меняется или сужается	Identity или ее прямоугольный фрагмент
Расширяется	Фрагмент Хадамарда с коэффициентом масштаба

Например, для слоя 784→2048 берем таблицу Хадамарда 2048×2048 и оставляем первые 784 столбца. Все коэффициенты умножаем на общий масштаб. В статье он равен √(2/M), в авторском коде - 1/√M, где M - размер полной таблицы [1,2]. Формула и код расходятся в √2 раза, поэтому я проверяю оба варианта.

У ResNet есть еще обходной путь: блок складывает вход и поправку F(x). Если последний слой ветки F начать с нулей, вход все равно пройдет дальше, а ветка сможет постепенно включиться. Поэтому нулевая остаточная ветка и полностью нулевая сеть ведут себя по-разному.

Авторы проверяли полносвязные сети, ResNet и Transformer. Например, ошибка ResNet-18 на CIFAR-10: ZerO 5,13±0,08%, Kaiming 5,15±0,13% (10 повторов, Table 2). Главный результат - работоспособный структурированный старт; разница в качестве здесь небольшая. Эксперименты с ResNet я не повторял.

2. Воспроизведение: MNIST

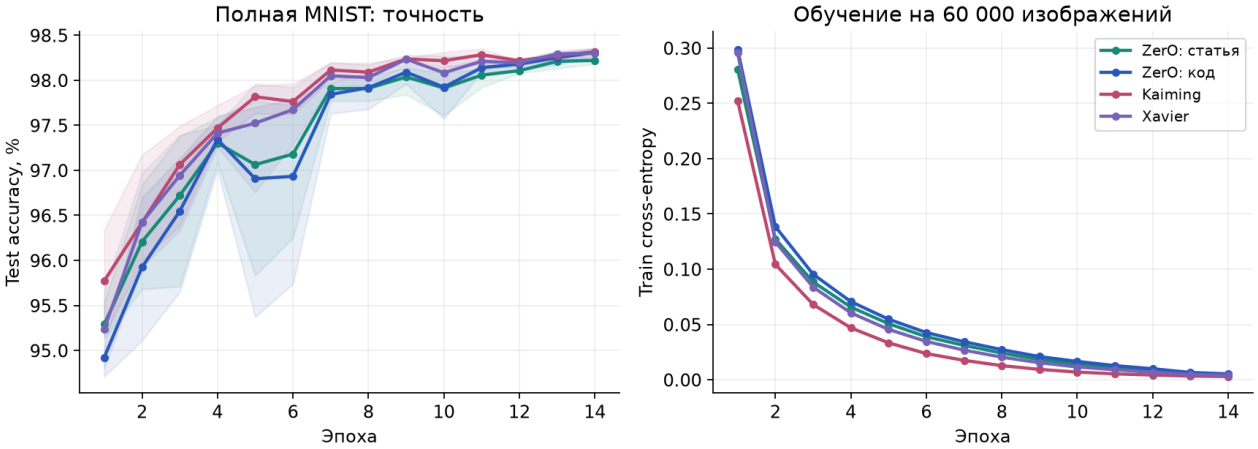
Я повторил опыт из Appendix B: распознавание рукописных цифр. В полной **MNIST** 60 000 обучающих и 10 000 тестовых картинок. Картинка 28×28 дает 784 числа яркости в [0,1].

Модель	784→2048→2048→10; ReLU; без bias
Обучение	14 эпох (полных проходов), SGD, шаг 0,1; по 64 картинки; momentum=weight_decay=0
Расписание	После эпохи 12 шаг умножается на 0,7
Повторы	По 3 для четырех стартов; по 1 для identity и всех нулей

Архитектура, 14 эпох и SGD взяты из статьи; batch, функция ошибки и расписание - из авторского notebook. В нем самом есть еще слой 784→784 и 20 эпох. Настройки, которых не хватало в статье, взял из авторского ноутбука.

Начальные веса	Точность, %
ZerO, масштаб статьи	98,22 ± 0,03
ZerO, масштаб кода	98,31 ± 0,04
Kaiming	98,32 ± 0,04
Xavier	98,30 ± 0,05

Kaiming и Xavier - обычные способы случайного старта. Точность - доля верных ответов на test после последней эпохи; ± - выборочное стандартное отклонение трех повторов, в процентных пунктах.

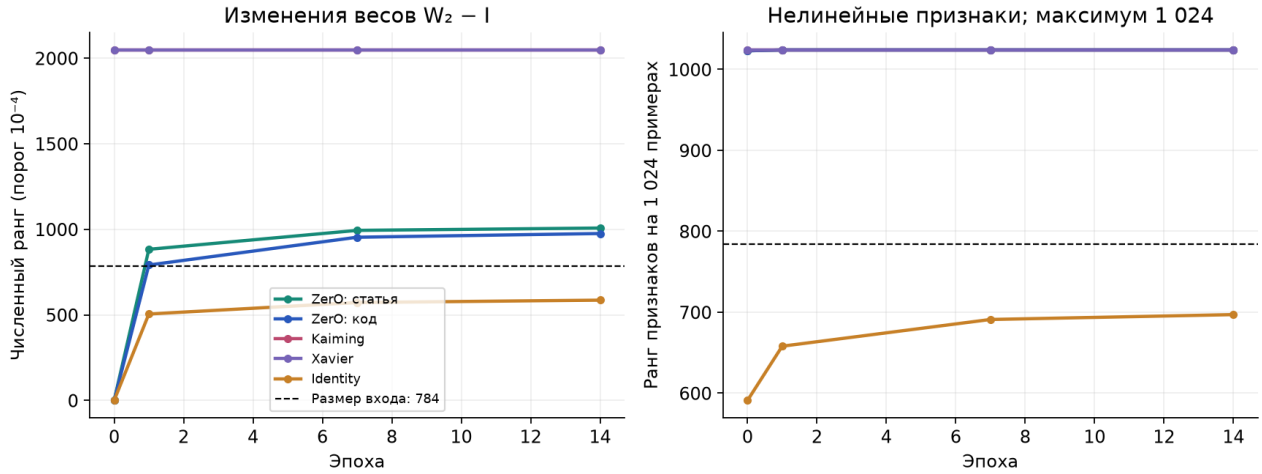


Собственные измерения. Линии - среднее; полосы - минимум и максимум трех повторов. Справа - функция ошибки обучения: меньше лучше.

Все четыре старта во всех повторах прошли 98%: **этот результат Appendix B удалось повторить.** Identity дал 98,22%; все нули - 9,80%. Нулевая сеть не учится и всегда выбирает класс 0.

3. А добавленная ширина работает?

Дальше я проверил механизм статьи. При identity и ZerO вторая скрытая матрица W_2 начинает с I. Сама I уже полного ранга, поэтому я измерял **выученную поправку $\Delta W = W_2 - I$**, а также ранг скрытых признаков.



Собственные измерения. Численный ранг при пороге 10^{-4}: слабые направления ниже порога не учитываются. Признаки измерены на фиксированных 1024 train-картинках.

У identity ранг поправки 586, у ZerO по статье - в среднем 1007,3. На этом пороге ZerO вышел за 784 направления, соответствующие размеру входа. Identity остался ниже, но получил 98,22% точности. **Ограниченный ранг не помешал хорошо решать эту задачу.**

Но есть нюанс. Признаки измерены только на 1024 картинках, поэтому ранг выше 1024 такая проверка не покажет. Еще компьютер должен отличать ноль от очень маленького числа. При пороге 10^{-2} ранг поправки ZerO около 338, а при 10^{-4} - уже 1007. Поэтому число без порога мало что говорит.

Что дало сравнение масштабов

Нормировка авторского кода дала 98,31% вместо 98,22%. Значит, расхождение формулы и кода здесь заметно в результате. Из этих повторов не следует, что один масштаб всегда лучше.

Вот что здесь интересно: ранг помогает понять, как устроены признаки. Но задача здесь - распознавать цифры. Сравнивать методы только по красивому значению ранга было бы ошибкой.

4. Дополнение: Transformer на цифрах

Картинку режем на 16 квадратов 7×7 - **патчей**. Каждый превращаем в набор признаков. Attention вычисляет, с каким весом один патч должен учитывать остальные; четыре головы делают это параллельно. После двух блоков усредняем признаки и предсказываем цифру.

Модель маленькая: ширина 64, внутренний слой 256, ReLU. Обучение - AdamW с шагом 0,001, без weight decay и dropout, по 128 картинок, 10 эпох. На полной MNIST сравниваю четыре старта, каждый по три повтора.

Начальные веса	Точность, %
Xavier	97,59 ± 0,59
Identity: копирование	96,97 ± 0,15
ZerO-inspired, статья	97,19 ± 0,11
ZerO-inspired, код	97,67 ± 0,27

Точность после 10 эпох; ± - выборочное стандартное отклонение трех повторов. Это собственные измерения, не результаты языковой задачи из статьи.

Что именно перенесено

Три матрицы attention превращают признаки в запросы Q, ключи K и передаваемую информацию V. Сходство запросов с ключами задает веса внимания. В **ZerO-inspired** матрица Q начинается с identity, K и V - с нулей; выходная матрица attention - identity. Остальные слои используют Algorithm 1, позиции патчей фиксированы.

На старте все ключи нулевые: каждому патчу достается одинаковый вес 1/16. Нулевые V пока обнуляют выход attention, а обходной путь передает признаки дальше. По журналам первым получает ненулевой сигнал для обновления V, затем K и Q. Ветка постепенно включается в обучение.

Вариант с масштабом кода выше Xavier всего на 0,09 процентного пункта в среднем. На трех повторах с таким разбросом объявлять победителя рано. Зато видно главное: **сеть обучилась с заранее заданными начальными весами.**

Transformer уже есть в статье на WikiText-2. Дополнение здесь - классификация цифр по патчам и диагностика обучения. Переносить результат маленькой модели на большие языковые модели оснований нет.

5. Выводы и как повторить

Мне в ZerO понравился сам вопрос: обязательно ли начинать со случайных весов? В этих опытах оказалось, что нет. Основной ZerO получил больше 98% на полной MNIST, адаптация обучилась в небольшом Transformer. По качеству случайные Kaiming и Xavier остались рядом.

Дополнительные проверки оказались полезнее поиска победителя в сотых долях процента. Identity хорошо решал задачу при ограниченном ранге. Масштаб весов менял результат. Порог менял оценку ранга. При этом одинаковые начальные веса ZerO не делают все обучение одинаковым: порядок пачек между повторами различается.

Еще одна проверка

В Lemma 2 / Appendix A.4 четыре вектора названы независимыми, хотя один выражается через остальные. Их ранг 3. В check_theory.py есть проверка и исправленный пример слоя 3→4 с четырьмя независимыми направлениями. Эта ошибка не отменяет результаты обучения [1].

Повторение работы

reproduce.ipynb содержит код и журналы всех 26 завершенных GPU-запусков. При RUN_TRAINING=False он проверяет сохраненные результаты и строит графики без GPU. Для нового обучения в Colab выбираем GPU и RUN_TRAINING=True; DOWNLOAD_RESULTS=True позволяет скачать новые журналы отдельно.

С чего начать читать код: initialization.py - начальные веса, models.py - модели, train.py - обучение. README.md содержит команды запуска. Настройки и версии сохранены в metadata.json.

Источники

[1] Zhao et al. ZerO Initialization, TMLR 2022 / arXiv v3. Algorithm 1; Table 2; Appendix A.4 и B.

[2] Авторский код, example_mnist.ipynb. Использован для проверки нормировки и настроек обучения.

[3] torchvision.datasets.MNIST. CSV/JSON измерений: results/paper, controls и transformer.